

实验 2 分析报告

内容：内存分配与回收、地址转换

一、宏和全局变量（补全，错误的修改）

宏：

LIST_HEAD 创建链表头，名称、数据类型都可以根据需求设置

LIST_HEAD_INITIALIZE 初始化链表头，可用于直接重置链表

LIST_ENTRY 链表第一个节点，其中 *le_next 指向下一个节点， **le_prev 指向前一个节点的 *le_next

LIST_EMPTY 判断链表是否为空 LIST_FIRST 返回指向链表第一个节点的指针

LIST_FOREACH 用于遍历链表 LIST_INIT 创建新链表 LIST_INSERT_AFTER 在某结点后插入新节点

LIST_INSERT_TAIL 在某结点前插入新节点，需要操作需要操作新节点两个指针，原前面结点的 le_next 指针，原后面结点的 le_prev 指针

LIST_INSERT_HEAD 在链表头插入新节点，需要首先判断原头结点是否为空，若为空，直接将新节点作为头结点，不空则需要设置原头结点的 le_prev 指针

LIST_INSERT_TAIL 在链表尾部插入新节点

LIST_REMOVE 移除节点，若被移除的节点位于链表尾部，则只需设置其前面节点的指针，否则还需要设置其后面节点的 le_prev 指针

PPN(pa)：这个宏用于从物理地址 pa 中提取页面号。

PGSHIFT：这个宏定义了页面大小的位移量，通常用于将页号转换为物理地址。

KADDR(pa)：这个宏用于将物理地址 pa 转换为内核虚拟地址。

PDE 和 Pte：这两个宏或类型定义用于表示页面目录项（Page Directory Entry）和页面表项（Page Table Entry）。

PDX(va) 和 PTX(va)：这两个宏用于从虚拟地址 va 中提取页面目录索引和页面表索引。

PTE_ADDR(pte)：这个宏用于从页面表项 pte 中提取物理页帧的地址。

PTE_V：这个宏用于检查页面目录项或页面表项中的有效位，通常表示页面是否存在。

npage：这个宏或全局变量用于表示系统中页面的总数。

panic：这个宏用于在出现严重错误时打印错误信息并停止程序运行。

ROUND：对齐函数，用于将一个值向上舍入到特定的边界。

BY2PG：定义了一页的大小，通常是 2 的幂，比如 4KB。

PDX(va)：从虚拟地址 va 中提取页面目录索引。

PTX(va)：从虚拟地址 va 中提取页面表索引。

PTE_ADDR(pte)：从页表项 pte 中提取物理页帧的地址。

PADDR(freemem)：将虚拟地址 freemem 转换为物理地址。

KADDR(pa)：将物理地址 pa 转换为内核虚拟地址。

PDE 和 Pte：分别定义了页面目录项和页面表项的数据类型。

PTE_V：页表项或页面目录项的有效位标志。

ULIM：用户空间的最大地址。

NENV：环境结构数组的大小。

UPAGES 和 UENVS：分别表示 pages 数组和 envs 数组的虚拟地址。

PADDR: 将虚拟地址转换为物理地址。
VA2PFN(va): 将虚拟地址 va 转换为页框号。
GET_ENV_ASID(envid): 获取环境的地址空间标识符 (ASID)。
PTE_R: 页表项的可读权限位。
page2pa: 将页面结构转换为物理地址。
pa2page: 将物理地址转换为页面结构。
va2pa: 将虚拟地址转换为物理地址。
tlb_out: 用于更新 TLB (Translation Lookaside Buffer)。
curenv: 指向当前环境的指针。
PTE_ADDR(va): 提取虚拟地址的物理地址部分。
PDMAP: 可能用于表示页面目录映射的特定虚拟地址。

全局变量:

```
static inline u_long  
extern struct Page *pages; // pages 是一个全局数组，其中包含了系统中所有页面的 Page 结构  
体。  
/*pmap.c 文件*/  
/* 这些变量由 mips_detect_memory() 设置 */  
u_long maxpa; /* 最大物理地址 */  
u_long npage; /* 内存总量，单位：页 */  
u_long basemem; /* 基础内存总量，单位：字节 */  
u_long extmem; /* 扩展内存总量，单位：字节 */  
Pde *boot_pgdir;  
struct Page *pages; //页控制块： pages 数组  
Pde *boot_pgdir;  
struct Page *pages; //页控制块： pages 数组
```

二、MIPS 虚存映射布局

三、分析记录内存情况和页表的数据结构分析

四、关于内存初始化

对 `mips_detect_memory()`; `mips_vm_init()`; `page_init()`; 函数每条语句进行注释，最后总结具体初始化流程

五、内存分配和释放

对 `mm/pmap.c` 中的 `page_alloc` 和 `page_free` 函数每条语句进行注释，总结具体的分配流程和回收流程。注意涉及的调用函数也要找出来具体注释

六、地址转换

首先根据实验指导书总结地址转换方法

TLB 重填：（这部分大家可以对下面内容进行修改）

当我们使用 `kuseg` 地址空间的虚拟地址访问内存时，我们会通过 TLB 将其转换为物理地址。当 TLB 中查询不到对应的物理地址时，就会发生 TLB Miss 异常。这时将跳转到异常处理函数，执行 TLB 重填。

TLB 的重填过程由 `kern/tlb_asm.S` 中的 `do_tlb_refill` 函数完成。该函数是汇编实现的。首先，定义了一个字的变量，标签为 `tlb_refill_ra` 流程。

在 MIPS 架构中，当发生 TLB（Translation Lookaside Buffer）缺失（TLB Miss）时，CPU 会触发一个异常，并跳转到异常处理程序。这个异常处理程序通常是用汇编语言编写的，因为它需要直接与 CPU 的硬件特性交互。汇编语言函数 `do_tlb_refill` 负责处理 TLB 缺失异常。这个汇编函数会执行一些基本的操作：

1. 当 CPU 遇到 TLB 缺失时，它跳转到汇编语言编写的 `do_tlb_refill` 函数。
2. `do_tlb_refill` 保存当前的寄存器状态，特别是返回地址寄存器 `ra`。
3. `do_tlb_refill` 设置必要的参数，通常是虚拟地址和地址空间标识符（ASID）。
4. `do_tlb_refill` 调用 C 语言函数 `_do_tlb_refill` 来处理 TLB 缺失。
5. `_do_tlb_refill` 执行必要的操作来找到或创建正确的 PTE。
6. `_do_tlb_refill` 返回找到的 PTE。
7. `do_tlb_refill` 使用返回的 PTE 来填充 TLB。
8. `do_tlb_refill` 恢复寄存器状态，并返回到触发异常的指令的下一条指令。

接着我们详细深入 `_do_tlb_refill`，这个函数在 `kern/tlbex.c` 中。正如 `hints` 所说，在这个函数中，我们会不断查找虚拟地址对应的页表项，如果未找到，则试图申请一个新的页表项。最终返回申请到的页表项的内容。

数据段：

这行代码定义了一个位于 `.data` 段的全局变量 `tlb_refill_ra`，它用来存储在执行 `do_tlb_refill`

函数期间的返回地址（ ra 寄存器的值）。

在 TLB 重填过程中，当发生 TLB Miss 异常时，CPU 会跳转到 do_tlb_refill 函数。这个函数需要知道异常发生前的返回地址，以便在完成 TLB 重填后能够正确地恢复执行流程。因此，tlb_refill_ra 变量用于保存这个返回地址，以便在 TLB 重填完成后可以返回到正确的位置继续执行。

```
.data
```

```
//在汇编语言中，.data 段用于声明初始化的数据或全局变量/*tlb_refill_ra 是一个变量名，它将被用来存储一个返回地址。*/
```

```
tlb_refill_ra:
```

```
//这是一个标签（label），它用于在汇编代码中引用特定的内存位置。
```

```
/*这表示在内存中分配一个字的存储空间，并将其初始化为 0。*/
```

```
.word 0
```

```
/*
```

这段代码在数据段（.data）中声明了一个名为 tlb_refill_ra 的变量，并将其初始化为 0。这个变量的作用是在 TLB 重填过程中存储调用 do_tlb_refill 函数之前的返回地址（即调用 do_tlb_refill 之前的程序计数器 PC 值）。

```
*/
```

代码段：

这行代码定义了汇编函数 do_tlb_refill，它是一个非叶函数，因为它可能会调用其他函数。首先我们使用 NESTED 定义函数标签。NESTED 与 LEAF 宏相对应。前者表示非叶函数，后者表示叶函数。

```
.text
```

```
NESTED(do_tlb_refill, 0, zero)
```

```
/*
```

NESTED 宏：这个宏用于定义一个非叶函数。非叶函数是指可能会调用其他函数的函数，因此它需要维护一个栈帧以便能够保存返回地址和可能需要保存的寄存器。

do_tlb_refill：这是函数的标签名，它将在汇编代码中引用这个函数的开始位置。

第一个参数 0：这通常表示函数的帧大小。在这个例子中，它被设置为 0，这可能意味着函数不需要为局部变量分配额外的栈空间，或者这个信息将被后续的代码覆盖。

第二个参数 zero：在 MIPS 汇编中，zero 是一个特殊的寄存器，其值始终为 0。在这里，zero 被用作一个占位符，因为 NESTED 宏可能期望有三个参数，但在这里不需要第三个参数，所以传递 zero。

```
/*
```

我们希望汇编尽可能少，因此希望 do_tlb_refill 只做必要的处理，随后调用 c 函数进一步处理。因此首先我们设置参数。第一个参数是 BadVAddr 寄存器的值，即发生 TLB Miss 的虚拟地址；第二个参数是 EntryHi 寄存器的 6-11 位。即当前进程的 ASID。

```
mfc0 a0, CP0_BADVADDR
```

```
mfc0 a1, CP0_ENTRYHI
```

```
srl a1, a1, 6
```

```
andi a1, a1, 0b111111
```

这里, 我们使用 `mfc0` 指令从协处理器 0(CP0)中读取 `BadVAddr` 和 `EntryHi` 寄存器的值。`BadVAddr` 寄存器包含了触发 TLB 缺失异常的虚拟地址, 而 `EntryHi` 寄存器包含了地址空间标识符 (ASID) 和其他信息。

我们将 `EntryHi` 寄存器的值右移 6 位, 然后与掩码 `0b111111` 进行位与操作, 以获取 ASID。在这里, 我们首先将当前的返回地址 (`ra` 寄存器) 保存到 `tlb_refill_ra` 变量中。然后, 我们跳转到 C 函数 `_do_tlb_refill`。调用完成后, 我们从 `tlb_refill_ra` 变量中恢复 `ra` 寄存器的值。

```
sw ra, tlb_refill_ra
jal _do_tlb_refill
lw ra, tlb_refill_ra
```

`_do_tlb_refill` 会返回虚拟地址对应的页表项。我们将该返回值存入 `EntryLo`, 并将 `EntryHi` 和 `EntryLo` 的值写入 TLB。

```
mtc0 v0, CP0_ENTRYLO0
// See <IDT R30xx Family Software Reference Manual> Chapter 6-8
nop
/* Hint: use 'tlbwr' to write CP0.EntryHi/Lo into a random tlb entry. */
/* Exercise 2.10: Your code here. */
tlbwr
jr ra
END(do_tlb_refill)
/*
```

执行 `_do_tlb_refill` 函数后, 返回值 (页表项) 将存储在 `v0` 寄存器中。我们使用 `mtc0` 指令将这个值写入 `CP0_ENTRYLO0` 寄存器。然后, 我们使用 `tlbwr` 指令将 `CP0.EntryHi` 和 `CP0.EntryLo` 寄存器的值写入一个随机的 TLB 条目中。最后, 我们跳回到原来的返回地址 `ra`, 完成 TLB 重填。

```
*/
```

这样就完成了 TLB 重填。跳回到正常程序后, 此前产生异常的虚拟地址就可以通过 TLB 访问内存了。

C 部分 `_do_tlb_refill`

接着我们详细深入 `_do_tlb_refill`, 这个函数在 `kern/tlbex.c` 中。正如 `hints` 所说, 在这个函数中, 我们会不断查找虚拟地址对应的页表项 (PTE), 如果未找到, 则试图申请一个新的页表项。最终返回申请到的页表项的内容。

```
Pte _do_tlb_refill(u_long va, u_int asid) {
    Pte *pte;
    while (page_lookup(cur_pgdir, va, &pte) == NULL) {
        passive_alloc(va, cur_pgdir, asid);
    }
    return *pte;
}
```

```
}  
/*
```

1.使用 `page_lookup` 函数在当前进程的页目录 `cur_pgdir` 中查找与虚拟地址 `va` 相关的 PTE。如果找不到，`page_lookup` 返回 `NULL`。

2.如果 `page_lookup` 返回 `NULL`，表示 PTE 不存在，因此我们调用 `passive_alloc` 函数来分配一个新的

物理页，并将其映射到虚拟地址 `va`。

3.这个过程会一直重复，直到 `page_lookup` 成功找到 PTE。

4.一旦找到 PTE，函数返回 PTE 的内容。

这个函数是 TLB 重填过程中的关键部分，因为它确保了虚拟地址到物理地址的正确映射。

```
*/
```

对 `mm/pmap.c` 中 `boot_pgdir_walk` 和 `pgdir_walk` 两个函数每条语句进行注释，总结页表具体创建流程。注意涉及的调用函数也要找出来具体注释

对 `boot_pgdir_walk` 函数 `pgdir_walk` 函数每条语句进行注释，总结具体的转换流程。注意涉及的调用函数也要找出来具体注释

对 `boot_map_segment` 函数每条语句进行注释，分析如何将制定的物理内存与虚拟内存建立起映射。注意涉及的调用函数也要找出来具体注释